Unidad 5: Almacenamiento y Procesamiento Distribuido

1. Sistemas de Almacenamiento y sus Propiedades

Antes de profundizar en los sistemas de archivos distribuidos, es útil ubicarlos dentro del panorama general de los sistemas de almacenamiento. La siguiente tabla compara sus propiedades clave:

Sistema	Compartición	Persistencia	Cache/Réplicas distribuidas	Consistencia	Ejemplo
Memoria principal	✗	✗	✗	1 (strict one-copy)	RAM
Sistema de archivos	✗	✓	✗	1	UNIX file system
Sistema de archivos distribuido	✓	✓	✓	✓	Sun NFS
Web	✓	✓	✓	✗	Web server
Memoria compartida distribuida	✓	✗	✓	✓	Ivy
Almacén de objetos persistentes	✓	✓	✗	1	CORBA Persistent Object Service
Sistema P2P	✓	✓	✓	2 (weaker)	OceanStore

2. Sistema de Archivos Distribuido (SAD)

2.1 Definición y Objetivos

Un **Sistema de Archivos Distribuido (SAD)** tiene como objetivo principal compartir datos entre usuarios ofreciendo **transparencia**: el acceso a archivos remotos debe ser indistinguible del acceso local.

Sus objetivos secundarios son:

- **Disponibilidad**: el sistema debe estar operativo ante fallos parciales.
- **Rendimiento**: comparable al de un sistema de archivos tradicional.
- **Tolerancia a fallos**: mediante replicación y funcionamiento degradado.

2.2 Conceptos Básicos

Modelo cliente-servidor: el SAD se estructura en dos roles. Los *servicios del sistema de archivos* son las operaciones ofrecidas a los clientes (abrir, leer, escribir, cerrar); los *servidores del sistema de archivos* son procesos que implementan esos servicios.

Transparencia: implica que las mismas operaciones funcionan tanto para accesos locales como remotos, brindando una imagen única del sistema de archivos. Un SAD también apunta a la *independencia de la posición*: el nombre de un archivo no necesita cambiar cuando el objeto cambia de ubicación física.

Rendimiento: un SAD introduce sobrecargas adicionales respecto a un sistema local: comunicación por red, protocolos y posible duplicación de copias. Por ello se busca eliminar cuellos de botella y escalar horizontalmente.

Tolerancia a fallos: se logra mediante replicación y modos de funcionamiento degradado.

3. Gestión de Nombres

3.1 Servicio de Directorio

El **servicio de directorio** traduce nombres simbólicos (orientados al usuario) a nombres internos del sistema. Un directorio es una estructura que asocia de forma única nombres de archivos con nombres internos.

Existen dos enfoques para su implementación:

- Los directorios son **objetos independientes** gestionados por un servidor de directorios (SD) separado.
- Los directorios son **archivos especiales**, combinando el servidor de archivos y el de directorios en uno solo.

3.2 Gestión de Nombres en el Sistema Operativo Distribuido

Un sistema operativo distribuido ofrece un **servicio uniforme de nombres** para todos los objetos. Propiedades clave:

- **Transparencia de posición**: el nombre del objeto no revela su ubicación física.
- **Independencia de posición**: el nombre no cambia si el objeto es migrado.
- Debe manejar réplicas y ofrecer nombres orientados al usuario.

3.3 Nombres en Dos Niveles

El espacio de nombres es jerárquico. Hay tres alternativas principales:

1. **Máquina:archivo** — no ofrece ni transparencia ni independencia.
2. **Montar el sistema remoto sobre la jerarquía local (ej. NFS)** — puede generar espacios de nombres distintos por máquina.
3. **Espacio de nombres único en todas las máquinas** — proporciona verdadera transparencia.

4. Servicio de Archivos

El servicio de archivos gestiona el acceso a los datos. Sus aspectos más importantes son: semántica de coutilización, métodos de acceso, caché de bloques, coherencia de caché y técnicas para mejorar el rendimiento.

4.1 Semántica de Coutilización

Una **sesión** es la serie de accesos que realiza un cliente entre un `open` y un `close`. La semántica de coutilización define el comportamiento cuando múltiples procesos acceden simultáneamente al mismo archivo.

Semántica UNIX (ideal pero difícil de implementar en distribuidos):

- Una lectura ve los efectos de todas las escrituras previas.
- El efecto de dos escrituras sucesivas es el de la última.
- Los procesos pueden compartir el puntero de posición.

Semántica de Sesión (usada en SAD):

- Los cambios a un archivo abierto son visibles sólo en el proceso que los realizó.
- Una vez cerrado el archivo, los cambios son visibles para las sesiones posteriores.
- Si dos sesiones sobre el mismo archivo terminan concurrentemente, la última prevalece.
- No existen punteros compartidos.

4.2 Problemas en un SAD

Retrasos: si un `read` que ocurre después de un `write` llega primero al servidor, obtendrá el valor previo al `write`.

Performance: con un único servidor, el rendimiento puede degradarse. La solución habitual es permitir que los clientes mantengan copias locales en caché. Sin embargo, esto introduce el problema de coherencia: un cliente puede modificar localmente un archivo mientras otro cliente lee la versión desactualizada del servidor.

4.3 Métodos de Acceso a Archivos

Modelo carga/descarga (upload/download):

- Se transfiere el archivo completo al cliente.
- Se almacena en memoria o disco local.
- Usa semántica de sesión.
- Alta eficiencia en transferencia, pero gran latencia en el `open` y posibles múltiples copias.

Modelo de servicios remotos:

- El servidor provee todas las operaciones sobre el archivo.
- Acceso por bloques bajo un modelo cliente/servidor.

Empleo de caché en el cliente: útil en ambos modelos para reducir la latencia y el tráfico de red.

4.4 Tipos de Servidores

Servidores con estado:

- Al abrir un archivo, el servidor almacena información de la sesión y entrega un identificador único al cliente.
- Ventajas: mensajes más cortos, mejor rendimiento, facilita lectura adelantada y consistencia.

Servidores sin estado:

- Cada petición es autocontenida (incluye archivo y posición).
- Ventajas: mayor tolerancia a crashes, no consume memoria del servidor para mantener estado.

5. Estructura de Servidores de Archivos y Directorios

Existen dos grandes enfoques de arquitectura:

- **Servidor combinado**: un único servidor gestiona tanto directorios como archivos.
- **Servidor separado**: existe un servidor de directorios y un servidor de archivos diferenciados. El cliente envía el nombre simbólico al servidor de directorios, que retorna un nombre binario (máquina + nodo_i) que el servidor de archivos comprende. La jerarquía de directorios puede distribuirse entre varios servidores.

6. Caché de Bloques

El caché de bloques mejora el rendimiento explotando el **principio de localidad de referencias**:

- **Proximidad temporal**: datos accedidos recientemente tienden a volver a usarse.
- **Proximidad espacial**: datos cercanos en el espacio de direcciones tienden a accederse juntos.

Técnicas asociadas:

- **Lecturas adelantadas (prefetch)**: mejora el rendimiento en accesos secuenciales.
- **Escrituras diferidas**: mejora el rendimiento de escritura.

Otros tipos de caché: caché de nombres, caché de metadatos del sistema de archivos.

6.1 Localización de la Caché

Ubicación	Beneficio principal
En el **servidor**	Reduce accesos a disco
En el **cliente** (disco local)	Reduce tráfico de red y carga del servidor; más capacidad, no volátil
En el **cliente** (memoria principal)	Más rápido; menor capacidad, volátil

6.2 Tamaño de la Unidad de Caché

Un mayor tamaño puede aumentar la tasa de aciertos y mejorar la utilización de la red, pero agrava los problemas de coherencia. En cachés grandes, conviene usar bloques de 8 KB o más; en cachés pequeñas, bloques grandes son menos adecuados.

6.3 Políticas de Actualización

Escritura inmediata (write-through):

- Buena fiabilidad.
- Las escrituras son más lentas (rendimiento similar al modelo de accesos remotos).

Escritura diferida (write-back):

- Escrituras más rápidas y menor tráfico en la red.
- Riesgo: los datos pueden perderse si el cliente falla antes de enviar al servidor.

6.4 Problema de Coherencia de Caché

Cuando múltiples clientes cachean el mismo archivo y uno lo modifica, los demás pueden quedar con copias obsoletas. El problema es especialmente crítico en:

- **Coutilización en escritura secuencial**: entornos y aplicaciones distribuidas.
- **Coutilización en escritura concurrente**: aplicaciones paralelas.

7. NFS — Network File System

7.1 Introducción

NFS fue introducido por Sun Microsystems en **1985** y se convirtió en el sistema de archivos distribuido más usado en la industria. Su objetivo principal es proporcionar **acceso transparente** a archivos remotos para programas cliente que se ejecutan en UNIX y otros sistemas.

Características clave:

- Cualquier equipo puede actuar como servidor y exportar sus archivos, o como cliente y acceder a archivos remotos. En la práctica se usan servidores dedicados.
- Diseñado para soportar heterogeneidad de hardware y sistemas operativos.

7.2 Versiones de NFS

Versión	Protocolo / RFC	Año	Transporte
v1 y v2	RFC-1094	1989	UDP
v3	RFC-1813	1995	UDP o TCP
v4	RFC-3530	2003	TCP
v4.1	RFC-5661	—	TCP + soporte paralelo a servidores distribuidos

Todas las versiones emplean **RPC y XDR** para la comunicación. Las versiones más recientes mejoran seguridad, almacenamiento en caché, bloqueo y eficiencia de mensajes.

7.3 Arquitectura NFS

NFS se integra al kernel de UNIX a través de un módulo de **Sistema de Archivos Virtual (VFS)**. El VFS distingue entre archivos locales y remotos, y traduce entre los identificadores de archivo independientes que usa NFS y los identificadores internos de UNIX.

Flujo en el cliente:

Aplicación → UNIX system calls → Virtual File System (VFS)

└── Archivo local → UNIX file system → Disco local

└── Archivo remoto → NFS client → RPC/XDR → RED → NFS server

El VFS realiza seguimiento de los sistemas de archivos disponibles localmente y de forma remota, y redirige cada solicitud al módulo apropiado.

7.4 Traducción de Nombres en NFS

- El VFS almacena una entrada por cada archivo abierto denominada **vnode**.
- Cada vnode apunta a un **inodo** local o a un **inodo remoto**.
- El cliente realiza la traducción del path componente a componente. Si un vnode apunta a un inodo remoto, la búsqueda se delega al servidor.
- El cliente mantiene una **caché de nombres** (caché de atributos) para acelerar las traducciones.

7.5 Acceso a Archivos en NFS

- Las transferencias se realizan en bloques de **8 KB**.
- Los bloques se almacenan en la caché del cliente.
- Los clientes realizan **lecturas adelantadas** de un bloque.
- Las escrituras se realizan localmente; los bloques se envían al servidor al llenarse un bloque de 8 KB o al cerrarse el archivo.
- Los servidores NFS son **sin estado** (stateless): no hay mecanismos de control de concurrencia para asegurar semántica UNIX estricta.

Para aproximarse a la semántica UNIX:

- Al abrir un archivo, se verifica en el servidor si fue modificado.
- Si un bloque lleva más de 3 segundos en la caché del cliente, se valida su vigencia.

7.6 Protocolo NFS

NFS ofrece un conjunto de RPCs para operar sobre archivos remotos:

- Búsqueda de un archivo en un directorio.
- Lectura de entradas de directorio.
- Manipulación de enlaces y directorios.
- Acceso a atributos de un archivo.
- Lectura y escritura de archivos.

7.7 Desafíos de Diseño

Los principales retos de un SAD como NFS son:

- Uso eficaz del almacenamiento en caché del cliente para alcanzar rendimiento similar al local.
- Mantenimiento de coherencia entre múltiples copias en caché.
- Recuperación ante errores del cliente o del servidor.
- Alto rendimiento para archivos de todos los tamaños.
- Escalabilidad.

8. Ejemplos de Sistemas de Archivos

Sistemas de archivos locales y distribuidos de referencia:

- **ext / ext2 / ext3 / ext4** — sistemas de la familia Linux (ext3 con journaling, ext4 con soporte de extents).
- **FAT** — File Allocation Table, usado en DOS y Windows.

- **NTFS** — usado en Windows NT y posteriores.
- **HFS** — Hierarchical File System, macOS antiguo.
- **ISO 9660** — CD-ROM y DVD-ROM.
- **DAOS** — Distributed Asynchronous Object Storage (Intel).
- **OceanFS** — Cluster Storage System.
- **MadFS** — Mobile Agent-Based Distributed Network File System.

9. MapReduce

9.1 Contexto: Analítica de Gran Escala

El procesamiento masivo de datos plantea problemas de escala difíciles de resolver con enfoques tradicionales. Ejemplos típicos de análisis a gran escala:

- Calcular la frecuencia de palabras en un corpus de millones de documentos.
- Contar cuántas veces los usuarios hicieron clic en un conjunto masivo de anuncios.
- **PageRank**: calcula la importancia de una página web en función de las páginas que la enlazan.

Antes de MapReduce, los análisis se realizaban en **SQL** (muy expresivo, pero sin una solución clara para escalar la ejecución) o **manualmente** (distribuyendo documentos entre nodos, pero con problemas de fallos, balanceo de carga, nodos lentos y sesgo de datos).

9.2 Definición y Modelo de Programación

MapReduce fue creado por Google en 2004 (Jeffrey Dean y Sanjay Ghemawat), inspirado en las funciones `map` y `reduce` del lenguaje LISP. Es un **modelo de programación paralelizable** que:

- Se aplica a una amplia clase de aplicaciones de análisis.
- No es tan expresivo como SQL, pero es mucho más fácil de escalar.
- Es un sistema *runtime* escalable, eficiente y tolerante a fallas.

El programador solo necesita implementar dos funciones:

- **Map(clave, valor) → 0, 1 o más pares (clave', valor')**: procesa cada elemento de entrada de forma independiente y emite pares intermedios.
- **Reduce(clave, lista_de_valores) → salida**: agrega todos los valores de una misma clave para producir un resultado global por clave.

El sistema se encarga de agrupar los pares intermedios por clave, distribuir el trabajo y manejar fallos.

9.3 Cómo Trabaja: Fases de Ejecución

La ejecución de un job MapReduce sigue tres fases:

Fase Map:

- Los datos de entrada (almacenados en un DFS) se dividen en *splits*.
- Cada Mapper recibe un split y aplica la función Map a cada par (clave, valor).
- Los resultados intermedios se escriben localmente en el sistema de archivos del nodo.

Fase Intermedia (Shuffle & Sort):

- El sistema lee los pares emitidos por todos los Mappers, los **ordena por clave** y los agrupa: `(clave, [lista de valores])`.
- Los pares agrupados se distribuyen entre los Reducers (usualmente mediante una función hash sobre la clave: `hash(clave) % N`).

Fase Reduce:

- Cada Reducer recibe todos los pares `(clave, lista_de_valores)` correspondientes a sus claves asignadas.
- Aplica la función Reduce para producir el resultado final, que se escribe en el DFS.

Ejemplo: Contar palabras

```
Map(docID, contenido):
  FOR each word w IN contenido:
    emit(w, 1)


Reduce(word, value-list):
  result = 0
  FOR each v IN value-list:
    result += v
  emit(word, result)
```

El Mapper emite `(palabra, 1)` por cada palabra. El sistema agrupa y ordena: `(palabra, [1,1,1,...])`. El Reducer suma la lista y emite `(palabra, total)`.

9.4 Arquitectura

El sistema MapReduce está orquestado por un nodo **Master** que:

- Interactúa con el **DFS Name Server** para conocer la ubicación de los datos.
- Asigna tareas de *Map* a los Workers.
- Asigna tareas de *Reduce* a los Workers.
- Monitorea el progreso y maneja reejecutar tareas fallidas.

Los Workers ejecutan las tareas Map o Reduce y almacenan los datos intermedios en su sistema de archivos local antes de que sean leídos por los Reducers.

9.5 Tolerancia a Fallas y Performance

Tolerancia a fallas:

- MapReduce usa **réplicas** del DFS para tolerar la pérdida de nodos.
- El mecanismo de **heartbeats** detecta Workers caídos o lentos.

- Las tareas asignadas a nodos fallidos son reasignadas a otros Workers.
- Se generan **tareas de respaldo** (*backup tasks*): la primera en terminar "gana"; los nodos lentos se tratan como si hubieran fallado.
- En la publicación original de Google se reportó que el sistema completó un job perdiendo 1600 de 1800 máquinas.

Performance:

- La mayoría de implementaciones explotan la **localidad de datos**: se asignan tareas Map a Workers que ya tienen los datos en su nodo DFS, evitando transferencias de red innecesarias.
- La redundancia de tareas también se usa para mejorar la performance global.

9.6 Encadenamiento de Jobs

Si bien el modelo parece restrictivo, se puede extender mediante **encadenamiento**: la salida de los Reducers (pares clave-valor) puede ser la entrada de un siguiente job MapReduce. Esto permite implementar pipelines de análisis complejos, aunque no cubre todos los tipos de análisis posibles.

9.7 Otras Aplicaciones

Aplicaciones típicas de MapReduce (del paper original de Google):

- **Grep distribuido**: la función Map emite una línea si coincide con un patrón; Reduce es una función identidad que copia la salida.
- **Recuento de frecuencia de acceso a URL**: Map procesa registros de solicitudes y emite `(URL, 1)`; Reduce suma todos los valores de la misma URL.
- **Temperatura máxima por ciudad**: Map emite `(ciudad, temperatura)` por cada registro; Reduce selecciona el máximo por ciudad.
- **PageRank** y otros algoritmos de grafos.

9.8 Implementaciones

Existen implementaciones de código abierto del modelo MapReduce:

- **Apache Hadoop**: la implementación open-source más extendida.
- **Apache Flink**: orientado a procesamiento en tiempo real y por lotes.
- **Apache Spark**: framework de análisis de datos que admite MapReduce y además una gama más amplia de modelos de programación (streaming, ML, SQL).

10. Conclusiones de la Unidad

Los sistemas de archivos distribuidos y los modelos de procesamiento distribuido como MapReduce abordan los desafíos fundamentales de compartir, almacenar y procesar datos a escala. Los principales aprendizajes de la unidad son:

- Un SAD busca transparencia, disponibilidad, rendimiento y tolerancia a fallos. El equilibrio entre estos objetivos define las decisiones de diseño.
- La caché de cliente es la herramienta clave para alcanzar rendimiento comparable al de un sistema local, pero introduce el desafío de la **coherencia de caché**.
- NFS es el ejemplo paradigmático de SAD: usa servidores sin estado, RPC/XDR, y el VFS para ofrecer transparencia de acceso en entornos heterogéneos.

- MapReduce abstrae la complejidad del procesamiento paralelo masivo: el programador define Map y Reduce, y el sistema resuelve la distribución, la tolerancia a fallas y el escalado.
- La combinación de un DFS (almacenamiento distribuido) con MapReduce (cómputo distribuido) es el fundamento de los sistemas de Big Data modernos (Hadoop, Spark).